

Documento de Diseño de Juego (GDD)

Microjuego Asteroids

Nombre y apellidos: Tamer El Nagare El Nagar

Asignatura: Fundamentos de Videojuegos (UPM)

1. Escenas

El proyecto contiene una **única escena 2D principal** que contiene el juego completo. El entorno visual está basado en el espacio exterior, con un fondo oscuro estrellado.

A nivel técnico, la escena funciona en un espacio 2.5D; aunque visualmente se aprecian sprites en un plano bidimensional, en la realidad se operan componentes 3D (Rigid Bodies y Colliders) restringidos a los ejes X e Y.

La escena también contiene scripts gestores (Menu y Pooling), límites de cámara, una interfaz menú que se activa con el botón Escape, y un punto de aparición central para la nave del jugador.

2. Assets y GameObjects

La arquitectura del microjuego se basa en la instanciación y gestión de los siguientes prefabs y objetos de escena:

GameObject / Prefab	Descripción y componentes
Nave (Player)	Representación del jugador. Posee un SpriteRenderer, un Rigidbody y un CapsuleCollider. Controla el movimiento y dispara instanciando balas desde un subobjeto Gun.
Meteorito (Enemy)	Enemigo principal. Prefab con Rigidbody (afectado por el Spawner, sin gravedad) y CapsuleCollider. Su destrucción desencadena la creación de fragmentos. Destruirlos otorga 1 punto por meteorito. Contiene el tag Enemy.
Fragmento (Fragment)	Versión reducida del meteorito original. Actúa como obstáculo secundario. Destruirlos otorga 3 puntos por fragmento. Contiene el tag Fragment.

Bala (Bullet)	Proyectil de defensa. Utiliza un BoxCollider que detecta impactos y actualiza la puntuación.
Pooling	Centraliza las colas de reciclaje (pooling) para balas, meteoritos y fragmentos.
Canvas (UI)	Interfaz gráfica. Contiene el texto de puntuación (Points), un panel translúcido para el estado de Pausa y los botones de Reanudar y Reiniciar.

3. Mecánicas

- **Movimiento:** El jugador controla la nave aplicando empuje frontal (thrust) y giro direccional (rotate). La mecánica principal incluye límites de pantalla: cuando la nave abandona los límites visuales de la cámara por un eje, reaparece inmediatamente en el borde opuesto conservando su velocidad y trayectoria.
- **Disparo:** La nave puede proyectar balas en la dirección frontal de su pico. El uso del arma es ilimitado, sujeto únicamente a la velocidad de pulsación del botón de Espacio.
- **Spawn de enemigos:** El spawner genera meteoritos en la parte superior de la pantalla en posiciones X aleatorias. La dificultad es dinámica; la variable spawnRatePerMinute aumenta progresivamente basado en el tiempo de supervivencia Time.deltaTime, llenando la pantalla de obstáculos de forma progresiva.
- **Fragmentación:** Al impactar una bala contra un meteorito principal, el objeto se desactiva y en su última posición se instancian múltiples Fragmentos. Se aplica una fuerza explosiva radialmente, dispersando los fragmentos para añadir caos a la zona del juego.
- **Pausa y reinicio:** Mecánica de interrupción del juego cambiando la variable global Time.timeScale con valor 0. Permite al usuario congelar la simulación física y decidir si retomar la partida o recargar la escena restaurando los valores por defecto y puntuación 0.

4. Scripts y patrones de software

El código fuente aplica modularidad y responsabilidad única. Destaca la implementación de Object Pooling, un requisito extra de la segunda iteración, fundamental para la optimización de memoria. En lugar de destruir y crear objetos constantemente (lo cual sobrecarga el recolector

de basura), los objetos se activan y desactivan (SetActive(true/false)) desde pools representado por colas (Queue) preinstanciadas.

Script	Función
Pooling.cs	Implementa el object pooling. Inicializa colas para los meteoritos, fragmentos y balas. Proporciona métodos Get() y Return()
Player.cs	Captura el input del usuario, procesa las físicas de movimiento mediante AddForce, calcula los bordes para el teletransporte y recarga la escena en caso de colisión (OnCollisionEnter)
EnemySpawner.cs	Controla el ritmo de respawneo. Calcula una posición límite superior e invoca a Pooling.Instance.GetEnemy(), aplicándole una velocidad vectorial negativa en Y constante para simular la caída (no utiliza gravedad)
EnemySplit.cs	Adjunto al meteorito. Expone el método público Explode(). Al invocarse, devuelve el asteroide al Pool e invoca pequeños fragmentos aplicando fuerza de dispersión vectorial en 360 grados.
Bullet.cs	Gestiona el movimiento del proyectil y detecta colisiones. Dependiendo del tag impactado (Enemy o Fragment), desencadena Explode() o devuelve el fragmento al pool, sumando la puntuación correspondiente.
PoolLifeTime.cs	Utiliza Invoke en OnEnable para asegurar que todo objeto (balas erradas, meteoritos que superen la parte inferior de la pantalla) retornen obligatoriamente al pool tras x segundos si no interactúan con nada.
PauseMenu.cs	Interrumpe la simulación al pulsar Escape y gestiona la activación del menú. Posee funciones enlazadas a los eventos OnClick() de los botones.

5. Interacción y Flujo de Juego

Controles de Usuario

- Eje Vertical (W / Flecha Arriba): Propulsión (thrust) aplicando fuerza hacia adelante.
- Eje Horizontal (A y D / Flechas Izq-Der): Aplica rotación a la nave.
- Espacio (Space): Acción de disparo. Instancia un proyectil.
- Escape (Esc): Pausa / reanuda el juego y muestra el menú.

Flujo de Partida

La partida inicia automáticamente al cargar la escena, con el jugador situado en el punto (0,0). La mecánica es de supervivencia y puntuación tipo arcade. El objetivo es disparar a la mayor cantidad de meteoritos y fragmentos posibles sin ser golpeado.

Los errores penalizan drásticamente: si la nave colisiona con un obstáculo (Enemy o Fragment), la partida finaliza abruptamente. La puntuación se reinicia a 0 y la escena se recarga desde cero, obligando al jugador a comenzar un nuevo ciclo.